Embedded Linux and Yocto: a self-guided course

Christoph Ungricht

Contents

1 Embedded Linux and Yocto: a self-guided course

A third course for the same starting point. Where the bare-metal and RTOS courses lived on the MKR1000 itself, this one is mostly about a **different machine** that sits next to it: a Linux-capable SoC running an OS you build.

Same Socratic style. The first three modules answer a question many beginners get wrong: *can I run Linux on my MKR1000?* The honest answer drives everything that follows.

1.1 Target facts you'll keep referring to

- **MKR1000 hardware**: ATSAMD21G18A, Cortex-M0+, 256 KiB flash, 32 KiB SRAM, no MMU.
- **Linux's bare-minimum hardware needs** (Module 1) -- you'll write these down as your first exercise.
- **Reference SBCs** for this course: Raspberry Pi 4/5 (4-cores Cortex-A72/A76, multi-GiB RAM) or BeagleBone Black (Cortex-A8, 512 MiB RAM). Both can host your Yocto builds *and* be your target.

- **Documents you will use heavily**:
  1. **Yocto Project Mega-Manual** -- one document, ~1000 pages, the single source of truth for Yocto/BitBake.
  2. **Linux kernel "Documentation/" tree** for the architectures and subsystems you touch.
  3. **The SoC datasheet** of whatever Linux-capable chip you pick (very different beast from the SAMD21 datasheet -- thousands of pages).

  Action 0: skim the Yocto Project Mega-Manual's Table of Contents. Don't read; just orient yourself. Note the major sections: BitBake, layers, recipes, BSP, SDK, dev-manual, ref-manual. You'll come back module by module.

1.2 Course map

Part	Module	Topic
I. Concepts	1	What Linux needs from hardware (MMU, RAM, storage)
	2	Can the MKR1000 run Linux? (the honest answer)
	3	Architectures and SoCs for embedded Linux
II. Adding Linux next to the MKR1000	4	Pairing the MKR1000 with a Linux SBC
	5	Heterogeneous SoCs and Linux remoteproc (loading firmware onto a sibling core)
	6	Device tree and overlays: how Linux finds your hardware
	7	Communication channels: UART, SPI, I2C, USB-CDC, Ethernet/WiFi
	8	Worked example: MKR1000 as smart sensor + Pi as host
III. Building an embedded Linux distribution	9	Distribution choices: Yocto, Buildroot, Debian, OpenWrt
	10	Yocto concepts: BitBake, layers, recipes, machines, distros
	11	Your first image on a Raspberry Pi (or in QEMU)
	12	Custom layers, recipes, devshell, and adding your own software
IV. Going further	13	SoMs, custom carrier boards, and when to leave the SBC
	14	Real-time Linux: PREEMPT_RT, Xenomai, and the boundary with the MCU
Appendices	A	Glossary
	B	Yocto / BitBake cheat-sheet
	C	Reading list and primary sources

1.3 Module 1 -- What Linux needs from hardware

Before asking "can chip X run Linux", you need to know what Linux *requires*.

1.3.1 1.1 The non-negotiables

1. **A memory management unit (MMU)** -- mainline Linux is built around virtual memory. It uses an MMU to give each process its own address space, to enforce kernel/user separation, and to implement demand paging. **An MMU is not optional for mainline Linux.**
   - (There is a fork called **uClinux** for MMU-less parts, but it is a niche, no longer in active mainline development, and you should not plan around it for a new project.)
2. **Enough RAM** -- in practice, ~32 MiB is the absolute floor for an ancient stripped kernel. A modern Linux system you'd actually develop on wants **256 MiB minimum**, comfortably **512 MiB+**.
3. **Enough storage for the rootfs** -- a minimal embedded root file system is 8--16 MiB; a comfortable Yocto image is 64--256 MiB; a Debian-flavoured system is 1--4 GiB.
4. **A CPU architecture the kernel supports** -- ARMv7-A, ARMv8-A (AArch64), x86_64, RISC-V (RV64), MIPS, PowerPC, others. **Cortex-M class CPUs are not in this list** for mainline Linux because they lack an MMU.
5. **Drivers for the chip's peripherals** -- boot loader (typically **U-Boot**), kernel drivers for the SoC's UART/MMC/USB/Ethernet/etc. This is what an SoC vendor's **BSP** (Board Support Package) provides.

1.3.2 1.2 The boot chain

A Linux SoC boots in stages, each more capable than the last:

```
On-chip ROM bootloader (factory)
  -> first-stage bootloader (e.g. MLO, SPL, BL2) from boot media
     -> second-stage bootloader (U-Boot, GRUB, syslinux) loads the kernel
        -> Linux kernel (zImage / Image / vmlinux) takes over the CPU
           -> kernel mounts initramfs / rootfs
              -> init (systemd / SysV / OpenRC) starts user space
                 -> getty, daemons, your application
```

Compare to your MKR1000's "reset vector -> Reset_Handler -> main" in two lines. The Linux boot chain is structurally similar (each stage hands control to the next) but each stage is orders of magnitude bigger.

> Action: write down the four-line "needs" list above on paper and annotate, next to each, what your MKR1000 hardware has. (Spoiler: nothing meaningful matches. That answers Module 2.)

1.4 Module 2 -- Can the MKR1000 run Linux?

No. It is worth being unambiguous and explaining why, because the "why" is half the value of this module.

1.4.1 2.1 Concrete reasons

- **No MMU.** Cortex-M0+ has no MMU. It optionally has an MPU (Memory Protection Unit, simpler) -- not enough for Linux's process model.
- **32 KiB SRAM.** Even uClinux wanted more than ten times this for a minimal system.
- **256 KiB flash.** A bare Linux kernel `Image` is 5--10 MiB before any rootfs.

- **Single core, ~48 MHz peak.** Linux on this would be unusably slow even if it fit.

1.4.2 2.2 What about uClinux / NOMMU Linux?

The kernel does retain some MMU-less support (most active on Cortex-R and certain Blackfin/SuperH ports, and there is partial Cortex-M support in research/forks). For learning *Linux as a discipline*, the answer is still "use an MMU-equipped target." The MMU-less route is a specialist niche.

1.4.3 2.3 What about putting an "OS-like" thing on the MCU?

You already did. The RTOS course's FreeRTOS, with `tasks`, `queues`, `semaphores`, is exactly the right-sized "operating system" for this hardware. RTOS and Linux solve overlapping problems at very different scales.

1.4.4 2.4 So how do you actually "add Linux"?

Two architectural choices:

1. **Replace** the MKR1000 with a Linux-capable board for the higher-level work, and either drop the MKR1000 or keep it as a peripheral.
2. **Pair** the MKR1000 with a small Linux SBC sitting next to it on the same PCB or breadboard. The MCU handles real-time I/O; the Linux side handles networking, storage, UI, and anything that benefits from a full OS.

Pairing is the more interesting architecture and what the rest of this course assumes.

1.5 Module 3 -- Architectures and SoCs for embedded Linux

A non-exhaustive map of what's out there, oriented at hobby-to-light- industrial use.

1.5.1 3.1 ARM Cortex-A (the dominant choice)

Class	Examples	Where they show up
Cortex-A7 / A8 / A9	Allwinner H3, NXP i.MX6, TI AM335x	older SBCs (BeagleBone Black uses AM3358)
Cortex-A53 / A55	Broadcom BCM2837/BCM2711, Rockchip RK3328, NXP i.MX8M Mini	mid-range SBCs (Raspberry Pi 3/4, many CM4 carriers)
Cortex-A72 / A76 / A78	Broadcom BCM2711/BCM2712, Rockchip RK3588	Raspberry Pi 4/5, Orange Pi 5, Radxa Rock 5
Cortex-A53 + M-class	NXP i.MX8M, ST STM32MP1, TI Sitara AM62x, Renesas RZ/G2	"AMP" or "heterogeneous" SoCs that pair A-cores running Linux with M-cores running an RTOS on one die

> Action: bookmark the "STM32MP1" and "NXP i.MX8M Mini" reference pages. These are the most interesting parts in the long run because they put Linux and an RTOS on the same chip -- exactly the pairing from Module 2.4 with one die instead of two boards.

1.5.2 3.2 RISC-V

- **SiFive HiFive Unmatched / Unleashed** (now discontinued/rare): the first practical RISC-V Linux boards.
- **StarFive VisionFive 2**: current affordable RISC-V Linux SBC.
- **T-Head TH1520**, **Allwinner D1**: newer SoCs with growing distro support.

RISC-V Linux works, but the ecosystem (drivers, BSPs, GPU support) is still maturing. Pick ARM for productivity; pick RISC-V for learning the architecture.

1.5.3 3.3 x86_64

Embedded x86 still exists -- Intel Atom, AMD Ryzen Embedded -- but the hobby/SBC market is overwhelmingly ARM. Choose x86 if you need PC-compatible hardware (existing PCIe peripherals, Windows compatibility, mature graphics drivers).

1.5.4 3.4 Hobby-friendly SBCs to consider

Board	SoC	RAM	Strengths	Weaknesses
Raspberry Pi 4 / 5	BCM2711 / BCM2712 (A72 / A76)	1--8 GiB / 4--16 GiB	Best community, Yocto/Buildroot well-supported, plenty of OSS	Closed boot ROM and GPU on older models
Raspberry Pi Zero 2 W	RP3A0 (A53 quad, ~1 GHz)	512 MiB	Tiny form factor; runs full Pi OS	Limited RAM
BeagleBone Black	AM3358 (A8)	512 MiB	Excellent for Yocto learning, ti-bsp very mature, two PRU real-time cores on-chip	Aging
BeagleV-Ahead / BeagleV-Fire	TH1520 / Microchip PolarFire	4 GiB	RISC-V learning	New, fewer guides
Radxa / Orange Pi / Banana Pi 5	RK3588	4--32 GiB	Powerful, AArch64	Vendor BSP quality varies; "blob" drivers
NXP i.MX8M Mini EVK	i.MX8M Mini	2 GiB	Industrial pedigree; vendor Yocto BSP	Costlier than hobby SBCs
STM32MP135-DK / MP157-DK	STM32MP1 (A7 + M4/M7)	512 MiB / 1 GiB	A-core + M-core on one die; ST's Yocto BSP is decent	ST-specific; smaller community

1.5.5 Recommendation for this course

Raspberry Pi 4 (or Pi 5) as your Linux target. Reasons:

- You probably already have one or can get one easily.
- Yocto, Buildroot, Debian, and Pi OS all support it well.
- Tons of online help when you get stuck.

- It works as both a **build host** (compile Yocto on the Pi if you must) and as the **target** -- though you'll have a vastly better time Yocto-building on a beefy x86_64 desktop.

If you want a more "industrial Linux + MCU" feel, **BeagleBone Black** is the more educational target because the AM3358 has on-die PRUs (small real-time cores) and ti's Yocto BSP is one of the cleanest reference implementations to read.

1.6 Module 4 -- Pairing the MKR1000 with a Linux SBC

Your physical setup, conceptually:

```
+-------------------+      bus       +--------------------+
|  Raspberry Pi 4   | <----wires---> |   MKR1000 (SAMD21)  |
|  - Linux          |                |   - bare metal /    |
|  - WiFi/Ethernet  |                |     FreeRTOS        |
|  - storage / UI   |                |   - real-time I/O   |
|  - your app       |                |   - WiFi-101 chip   |
+-------------------+                +--------------------+
```

Why this split is interesting:

- **MCU**: deterministic timing, low-latency interrupt handling, low-power, owns the hardware peripherals it's wired to.
- **Linux side**: full networking stack, large storage, a process model, your favourite language ecosystem.

You decide where the boundary lives based on which side each task naturally belongs to. Some examples:

Task	MCU	Linux
50 kHz motor control loop	yes	no -- too jittery
HTTPS to a cloud service	no	yes
Reading an SPI ADC at 10 kHz	yes	maybe but messy
Storing 6 months of telemetry	no	yes
Driving a colour LCD with HDMI	no	yes

1.6.1 Physical interfaces

- **UART**: simplest. Pi GPIO 14/15 (TX/RX) -> MKR1000 RX/TX (SERCOM in UART mode). 3.3V on both sides; **no level shifter needed** for Pi <-> MKR1000.
- **SPI** with Pi as master: faster than UART, useful when the MCU is a sensor frontend.
- **I2C**: medium speed, multi-drop. Pi as master is convenient when multiple MCUs share the bus.
- **USB**: cable between Pi USB host and MKR1000 USB device. The MCU enumerates as a USB-CDC ACM serial device on the Pi (which is basically how the bootloader already works).
- **Ethernet / WiFi**: high speed, fully decoupled. The MCU runs a small HTTP server (Module 14 of the bare-metal course); the Pi is just another network client.

1.6.2 Voltage and grounding

- Pi GPIO is **3.3V tolerant only**; MKR1000 GPIO is **3.3V**. They are directly compatible.
- **Tie the grounds together** -- always. Floating grounds will appear to work and then fail intermittently.

Action: pick one channel (start with UART) and prove a "ping" round trip. Pi sends `"hello\n"`, MCU echoes `"world\n"`. Until that works, nothing more ambitious will.

1.7 Module 5 -- Heterogeneous SoCs and Linux remoteproc

Module 4 paired **two boards** -- the MKR1000 and a Pi -- talking over UART/SPI/USB. There's a second, sometimes-better architecture: **one chip with two kinds of CPU on the same die**. The Linux side runs on Cortex-A cores; an MCU-class sibling (Cortex-M, Cortex-R, or a smaller core) sits on the same SoC and runs your real-time firmware. Linux loads, starts, stops, and talks to that sibling through a kernel subsystem called **remoteproc**.

Crucially, **remoteproc cannot flash the MKR1000 from a Pi**. The SAMD21 is a different physical chip on the other end of a USB/SPI cable; it is not a sibling core of the Pi's BCM2711. But the *idea* of the MKR1000's blink translates directly to "blink firmware running on the M4 core of an STM32MP1" -- and *that* is exactly what remoteproc loads.

This module is conceptual + a worked example sketch. You don't need the hardware to read it; you do need it to actually run the example.

1.7.1 5.1 What remoteproc is, and what it isn't

Remoteproc = "remote processor framework" in the mainline Linux kernel (`drivers/remoteproc/`). It's the generic infrastructure for:

1. **Loading** a firmware image (an ELF, with a *resource table*) onto a non-Linux sibling core from Linux user space.
2. **Starting and stopping** that core (`echo start > .../state`).
3. **Establishing shared-memory communication** with it through a companion subsystem called **rpmsg** (virtio rings over the shared memory; comes for free once the firmware declares `vdev` resources in its resource table).

What remoteproc *is not*:

- Not a way to talk to an **external** chip over a cable. For that you use bossac (USB), OpenOCD (SWD), or a custom flasher.
- Not a hypervisor. The sibling core runs autonomously after start; Linux doesn't share its CPU.
- Not magic for any board that *has* a coprocessor. The SoC vendor has to ship a remoteproc *driver* for the specific MCU core. The generic framework is in mainline; the per-SoC piece comes from the BSP.

1.7.2 5.2 SoCs where this actually applies

SoC	Linux cores	Sibling core(s)	remoteproc support
STM32MP1 (ST)	1-2x Cortex-A7	1x Cortex-M4 (MP15) or M33 (MP25)	Mature; ST's BSP is open and well documented
NXP i.MX8M / 8M Mini / 8M Plus	2-4x Cortex-A53	1x Cortex-M4 (Mini/Nano) or M7 (Plus)	NXP BSP + mainline support
NXP i.MX RT11xx	1x Cortex-M7	1x Cortex-M4	Yes

SoC	Linux cores	Sibling core(s)	remoteproc support
TI Sitara AM62x / AM64x / AM65x	1-4x Cortex-A53/A72	Cortex-R5F + (on some) Cortex-M4 + PRUs	TI's mature BSP; mainline is catching up
TI AM335x (BeagleBone Black)	1x Cortex-A8	2x PRU (200 MHz real-time RISC cores)	Yes; PRU programming uses a unique ISA
Renesas RZ/G2L / RZ/V2L	2x Cortex-A55	1x Cortex-M33	Renesas BSP
Allwinner D1	1x RISC-V C906	small auxiliary core	Limited

The Raspberry Pi 4/5 has **no remoteproc sibling**. Its BCM2711/2712 SoCs are Cortex-A only -- you cannot use remoteproc on a Pi. If you want to play with this on hardware, the **STM32MP135-DK** or **TI BeagleBone Black** are the cheapest entry points.

1.7.3 5.3 What a firmware image for remoteproc looks like

Almost the same as your bare-metal blink, with three additions:

1. **Built for the sibling core's ISA**, not the Linux side. If the sibling is Cortex-M4, you compile with `-mcpu=cortex-m4 -mthumb` (you'd reuse 90% of your bare-metal toolchain knowledge).
2. **Linked against the sibling core's memory map**, which is typically a few **carveouts** in the SoC's main DDR that Linux reserves for the M-core, plus on-chip SRAM/TCM. The DT describes these.
3. **Contains a resource table**: a special ELF section, `.resource_table`, holding a C struct that tells remoteproc:
   - Which memory carveouts the firmware needs (size, expected virtual and physical address).
   - Whether it wants an rpmsg channel (and if so, the virtio device descriptor including ring sizes).
   - Optional trace buffer for `dmesg`-style logs visible from Linux.

The resource table is the only "new" concept versus your bare-metal build. Without one, remoteproc rejects the firmware.

A minimal resource table for blink (no rpmsg, no DDR -- just SRAM):

```
/* resource_table.c, linked into a .resource_table section. */
#include "remoteproc.h"

struct my_resource_table {
    struct resource_table base;
    uint32_t offset[1];
    struct fw_rsc_carveout sram;
} __attribute__((packed));

__attribute__((section(".resource_table")))
struct my_resource_table resource_table = {
    .base = {
        .ver       = 1,
        .num       = 1,            /* one resource follows */
        .reserved  = {0, 0},
    },
    .offset = { offsetof(struct my_resource_table, sram) },
    .sram = {
```

```
        .type     = RSC_CARVEOUT,
        .da       = 0x10000000,   /* device-side virtual addr */
        .pa       = 0,            /* let remoteproc allocate */
        .len      = 0x10000,      /* 64 KiB */
        .flags    = 0,
        .name     = "M4_SRAM",
    },
};
```

The structs are defined in the kernel header `include/linux/remoteproc.h`; vendor SDKs ship matching userspace headers. Don't write these from scratch -- start from your SoC vendor's example.

1.7.4 5.4 The blink-on-an-M4-via-remoteproc walkthrough

This is the analogue of "flash blink onto MKR1000 with bossac" -- but for an STM32MP1 dev board. Same idea, different boundary.

1.7.4.1 a. Build the M4 firmware Cross-compile your blink for the M4. Toolchain is the same `arm-none-eabi-gcc` you've been using:

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfloat-abi=hard \
    -ffreestanding -nostdlib -nostartfiles \
    -T m4-mp1.ld \
    startup.s main.c resource_table.c \
    -o blink-m4.elf
```

`m4-mp1.ld` is the M4 side's linker script (carveouts at addresses the resource table promised). ST publishes a template in their `OpenAMP_M4` examples.

1.7.4.2 b. Copy the ELF to the Linux side Either by including it in your Yocto image (Module 12) at `/lib/firmware/blink-m4.elf`, or by `scp`ing it after the fact:

```
scp blink-m4.elf root@stm32mp1:/lib/firmware/
```

1.7.4.3 c. Tell remoteproc which firmware to load

```
# On the STM32MP1, find your remoteproc:
ls /sys/class/remoteproc/
#   remoteproc0    <- the M4

# Point it at the firmware:
echo blink-m4.elf > /sys/class/remoteproc/remoteproc0/firmware

# Start it:
echo start > /sys/class/remoteproc/remoteproc0/state

# Stop it:
echo stop > /sys/class/remoteproc/remoteproc0/state
```

That's it. The LED wired to the M4's GPIO blinks. The Linux side is untouched and continues running its userspace normally.

1.7.4.4 d. (Optional) Add rpmsg so Linux and M4 can chat

Declare a `RSC_VDEV` virtio device in the resource table -- a virtio-ring carveout shared between the two sides. After `start`, a `/dev/rpmsg_ctrl0` device appears on Linux; on the M4 side, ST's OpenAMP middleware provides a matching API. From Linux you'd then `open()` a channel, `write()` bytes, and the M4's firmware receives them via OpenAMP callbacks.

This is the rough mental model: - **remoteproc** = loader and lifecycle control. - **rpmsg** = bytes between Linux and the sibling, layered on a shared-memory virtio queue. - **OpenAMP** = the C library on the sibling side that implements the other half of rpmsg.

1.7.5 5.5 Why this matters for your MKR1000 journey

You can't use remoteproc on a Pi-plus-MKR1000 setup, but the patterns transfer:

- **Same firmware mental model**: vector table, linker script, startup, register pokes. The bare-metal course's Modules 5-11 are the literal skill set.
- **Same "MCU runs real-time, Linux handles everything else" split.** The boundary just moves from "two boards over UART" to "two cores over shared memory."
- **When to migrate from MKR1000 + Pi to a single STM32MP1 / i.MX8M Mini**: when the latency / pin-count / bill-of-materials of two chips becomes worse than the complexity of one heterogeneous chip.

  Action: install Yocto's `meta-st-stm32mp` and build `core-image-minimal` for the `stm32mp1-disco` machine (Module 12). Without buying the board, you'll already have produced an image containing the right remoteproc kernel modules. Then, *when* you get the board, you do the four-line dance above and your blink runs on its M4. The skills are the same skills.

1.7.6 5.6 Common confusions

- **"Why can't I remoteproc-load my MKR1000?"** Because remoteproc operates over the SoC's internal bus -- the MCU has to be a silicon-level sibling of the Linux CPU. An external chip on USB is not a sibling; it's a peripheral. Use bossac instead.
- **"What about ESP-Hosted / RPMsg-Lite / OpenAMP on Pi?"** Those are ad-hoc layered protocols on top of UART/SPI links, mimicking the rpmsg API in user space. They are not the same as the kernel's remoteproc subsystem, and they have nothing to do with loading firmware -- they assume the peripheral is already running.
- **"Why is my resource table missing in the loaded ELF?"** Linker script forgot to include the section, or `--gc-sections` discarded it. Wrap it in `KEEP(*(.resource_table))` in the linker script.

1.8 Module 6 -- Device tree and overlays: how Linux finds your hardware

You wired the Pi's TXD to the MKR1000's RXD in Module 4. You boot the Pi. You type `ls /dev/tty*` and ... the UART you expect isn't there, or it is but `picocom` produces nothing. Welcome to **device tree** -- the concept that most trips up people coming from Arduino, where peripherals just "exist."

1.8.1 6.1 Why device tree exists

In the Arduino/MCU world your firmware *knows* the SoC's register addresses. They're in your header file. The CPU has one job and one configuration.

A Linux kernel has the opposite problem: the same compiled kernel binary boots on a Raspberry Pi 4, a BeagleBone, an i.MX8M board, and a hundred other systems built around dozens of different SoCs and PCB designs. The kernel cannot have any of those wired into the source. It needs a **runtime description of the hardware**: "this board has a UART at address X with IRQ Y connected to pins A and B; an I2C controller at address Z with these clients at addresses N, M; ..."

That description is the **device tree**. It's a structured text file (`.dts`, "device tree source"), compiled by `dtc` (device tree compiler) into a binary blob (`.dtb`, "device tree blob") that the boot loader hands to the kernel on every boot. The kernel walks the tree, finds drivers matching each "compatible" string, and binds them to the hardware described.

You can read your running Pi's effective device tree:

```
ls /proc/device-tree/
cat /proc/device-tree/soc/serial@7e201000/compatible
```

The filesystem at `/proc/device-tree` is the live tree the kernel booted with. Each directory is a node; each file is a property.

1.8.2 6.2 What an overlay is

A **device tree overlay** (`.dts` source that compiles to a `.dtbo` blob) is a small fragment that **patches** the base device tree at boot. Overlays let you toggle and configure hardware **without rebuilding the kernel or the base DT**:

- "Enable I2C bus 1." (off by default on the Pi to save pins.)
- "Disable Bluetooth so the primary UART is freed for your use."
- "Add an SPI display connected on bus 0, chip-select 0, with this reset pin."
- "Add an EEPROM at I2C address 0x50."

Without overlays you'd ship a different base DT for every board variant or every peripheral combination -- unmanageable. With overlays, the base DT describes the SoC and PCB, and overlays describe the optional peripherals.

Overlays compose: you can stack several on one boot.

1.8.3 6.3 The Raspberry Pi's overlay system: the easiest case

The Raspberry Pi firmware reads `/boot/firmware/config.txt` at boot, loads the base DT for the model, then loads any overlays you've listed. Hundreds of pre-built overlays ship with Raspberry Pi OS in `/boot/firmware/overlays/`. Each has a matching README entry in `/boot/firmware/overlays/README` describing its parameters.

Typical lines you'd add to `config.txt`:

```
# Disable the Bluetooth chip so the primary PL011 UART is on GPIO 14/15
dtoverlay=disable-bt

# Enable I2C bus 1
dtparam=i2c_arm=on

# Enable SPI bus 0
dtparam=spi=on

# Add a DS18B20 1-wire temperature sensor on GPIO 4
```

```
dtoverlay=w1-gpio,gpiopin=4

# A specific overlay with parameters
dtoverlay=spi1-1cs,cs0_pin=18
```

Reboot, and the kernel sees the new hardware. **No kernel rebuild, no recompile, no reflashing.**

1.8.4 6.4 Practical example: free the Pi's UART to talk to the MKR1000

Concrete scenario from Module 4: Pi GPIO 14/15 to MKR1000 RX/TX.

On a Pi 3, 4, or 5, the SoC has two UARTs available on the header: the high-performance **PL011** (`uart0`) and a simpler **mini-UART** (`uart1`). By default, the PL011 is bound to the on-board Bluetooth chip, so what appears on GPIO 14/15 is the mini-UART -- which has weird baud-rate behaviour because its clock is tied to the VPU frequency.

You want the PL011 on GPIO 14/15. Two options:

Option A: keep Bluetooth, swap UARTs.

```
# /boot/firmware/config.txt
dtoverlay=miniuart-bt
```

This puts the mini-UART on Bluetooth and the PL011 on GPIO 14/15. You also need to tell the BT driver about its lower clock; the overlay's README explains the dance.

Option B: kill Bluetooth, give all of GPIO 14/15 to the PL011.

```
# /boot/firmware/config.txt
dtoverlay=disable-bt
enable_uart=1
```

Then disable the serial-console getty that would otherwise grab the UART:

```
sudo systemctl disable serial-getty@ttyAMA0.service
```

Reboot. Now `/dev/ttyAMA0` is the PL011 on GPIO 14/15. `picocom -b 115200 /dev/ttyAMA0` talks directly to your MKR1000.

> Action: do this on your Pi. Pi `echo hello > /dev/ttyAMA0`. With a wire to the MKR1000's RX and your bare-metal UART driver running on the MCU, you should see "hello" on the MKR1000's debug LED, screen, or wherever you echoed it. This is the first time the two boards *talk*.

1.8.5 6.5 Writing your own overlay

When you need a peripheral that nobody has shipped an overlay for, you write one. The minimum useful file:

```
// my-eeprom.dts
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2711";  // Pi 4 SoC; match your target

    fragment@0 {
```

```
        target = <&i2c1>;
        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";

            eeprom@50 {
                compatible = "atmel,24c32";
                reg = <0x50>;
                pagesize = <32>;
            };
        };
    };
};
```

Read this top-down:

- `/dts-v1/; /plugin/;` -- header tags saying "this is device-tree source, version 1, intended as a plugin/overlay."
- `fragment@N` -- one patch to apply. Multiple fragments stack inside one overlay.
- `target = <&i2c1>` -- "patch the node labelled `i2c1` in the base tree." That label has to exist in the base DT; you find it by looking at the SoC's main `.dts` (in the kernel tree under `arch/arm/boot/dts/` or `arch/arm64/boot/dts/`).
- `__overlay__` -- the contents to splice in under `target`.
- `compatible = "atmel,24c32"` -- the magic string the kernel matches against driver tables. Every kernel driver registers a list of these.
- `reg = <0x50>` -- the device's address on the parent bus (here the I2C slave address).

Compile:

```
dtc -@ -I dts -O dtb -o my-eeprom.dtbo my-eeprom.dts
sudo cp my-eeprom.dtbo /boot/firmware/overlays/
echo "dtoverlay=my-eeprom" | sudo tee -a /boot/firmware/config.txt
sudo reboot
```

After boot, `i2cdetect -y 1` should show the device at 0x50, and the `at24` driver should bind to it; `ls /sys/bus/i2c/devices/` lists it.

> Action: pick any I2C device you have lying around (a real EEPROM, a temperature sensor, a tiny OLED), find its kernel driver's `compatible` string in the kernel source, write an overlay for it, and watch the kernel bind it without a single line of C from you. This is the "magic" of mainline Linux drivers.

1.8.6 6.6 Where overlays appear in Yocto (forward reference to Modules 10-12)

When you build a Yocto image (Module 11), the kernel's device-tree sources, the BSP-provided overlays, and any *project-specific* overlays from your layer get compiled and copied to the boot partition. In recipes you typically:

- list base DTBs to build via `KERNEL_DEVICETREE` in your machine config,
- ship overlay sources in your own layer and reference them similarly,
- adjust the boot-loader configuration (`u-boot.txt`, `extlinux.conf`, or `config.txt` on Pi) to load them.

The mental model is identical to the Raspberry Pi OS case: the kernel binary is generic, the device tree describes your specific hardware, overlays toggle and configure peripherals at boot.

1.8.7 6.7 Common confusions, briefly

- **"I edited the device tree, why isn't it taking effect?"** -- the base DT lives in the kernel package's boot partition files (`bcm2711-rpi-4-b.dtb` etc.). Overlays live separately under `overlays/`. Make sure you edited the right thing and that `config.txt` actually references it.
- **"Why is there a `status = "disabled";` node I have to enable?"** -- the SoC's full DT describes *every* peripheral the silicon has, but most are disabled because not every board uses them. Enabling one is a status flip in an overlay.
- **"What's the difference between a `dtparam` and a `dtoverlay`?"** -- on the Pi, `dtparam=foo=bar` sets a parameter on the base DT (e.g. `dtparam=i2c_arm=on` enables I2C1). `dtoverlay=name` loads a whole overlay file. The line between them is a Pi-firmware convenience; underneath, both ultimately patch the tree.
- **"On my non-Pi board the boot loader is U-Boot, where do overlays go?"** -- U-Boot has explicit commands (`fdt apply`, `fdt addr`) to apply overlay blobs. Distros that target U-Boot platforms typically use `extlinux.conf` with a `fdtoverlays` line or scripted `boot.scr`. Same idea, different glue.

1.9 Module 7 -- Communication channels in depth

The choice of channel affects everything downstream. Quick decisions:

Channel	Throughput	Complexity	When to pick
UART	0.1--3 Mbps	trivial	First prototype; simple command/response; logging
SPI	1--40 Mbps	moderate (master/slave roles, framing)	High-bandwidth sensor streams; binary protocols
I2C	0.1--3.4 Mbps	moderate	Multi-drop; registers-on-a-bus style
USB-CDC	~1 Mbps effective	low (Pi side: `/dev/ttyACM0` just works)	When you also want to power the MCU from the Pi over the same cable
Network (HTTP, MQTT, gRPC)	as fast as the link	high (you write protocol logic)	Production architectures; decoupled deployment

1.9.1 Protocol on top of the wire

The wire moves bytes. You still have to design a protocol:

1. **Framing**: how does the receiver know where one message ends and the next begins? Common: length prefix; sentinel byte (`\n` or COBS-encoded zero); time-gap.
2. **Encoding**: text (CSV, JSON), binary (raw struct, **Protocol Buffers**, **CBOR**, **MessagePack**).

3. **Integrity**: CRC-16/32 on each frame. Cheap to compute, catches the overwhelming majority of line errors.
4. **Acknowledgement**: ACK/NAK per frame? Sliding window? Or fire-and-forget?
5. **Versioning**: include a protocol version byte from day one. You will regret it if you don't.

For learning, **newline-delimited JSON over UART** is the right starter: human-readable, debug-friendly with `picocom`, terrible for high bandwidth -- which makes its limits visible.

1.9.2 A note on `screen` and `picocom`

On the Linux side, your first debug tool will be:

```
picocom -b 115200 /dev/ttyAMA0
```

or, on the Pi specifically, also `minicom` and `screen`. Get comfortable enough with one of them that opening a serial port is reflex.

1.10 Module 8 -- Worked example: MKR1000 as smart sensor, Pi as host

Concrete project to do alongside this course. By the time you finish Module 12 you'll be running this whole stack on Yocto-built Linux.

1.10.1 Concept

- **MKR1000** reads a temperature sensor (e.g. a one-wire DS18B20 or any I2C temp sensor on its SERCOM) once per second and sends `{ "t_c": 21.4, "seq": 12345 }\n` lines over UART to the Pi.
- **Pi** runs a Python (or Rust, Go, your pick) daemon that:
  - reads lines from `/dev/serial0`,
  - stores them in a small SQLite database,
  - exposes an HTTP endpoint `GET /latest` returning the most recent reading,
  - exposes a Prometheus-style `/metrics` endpoint.
- A browser on your laptop talks to the Pi over WiFi.

1.10.2 Why this split

The MKR1000 owns the sensor's timing-critical bit-banging and a 1 Hz sample loop -- well within its comfort zone. The Pi owns everything to do with the network and the database -- well within Linux's. Neither side does both, and the wire between them is a single character-stream of 30 bytes/second. This is the canonical embedded Linux + MCU pattern.

> Action: do this exact project in plain Raspberry Pi OS first. Modules 9-12 will then take you through rebuilding the same Pi with a custom Yocto image so you can ship it.

1.11 Module 9 -- Distribution choices

A "distribution" is a kernel + libc + userspace + tooling, packaged for installation. For embedded, the credible options divide cleanly:

1.11.1 9.1 Ready-made distributions

- **Raspberry Pi OS** (Debian-based): trivial to start; comfortable for hobby work; not what you'd ship as a tightly controlled product.
- **Ubuntu / Debian** (mainline): same idea, broader hardware support, larger images.
- **Armbian**: Debian/Ubuntu-derived, focused on ARM SBCs that upstream Debian doesn't support well.
- **Alpine Linux**: musl + busybox, very small (50 MiB rootfs is ordinary), Docker-flavoured ergonomics. Excellent for headless embedded.

Use these when you want a working Linux right now and don't care about custom packaging. **Stop here if Module 8's project is your endpoint.**

1.11.2 9.2 Build-it-yourself distribution generators

- **Buildroot**: a Makefile-driven, monolithic build system. One config -> one image. Fast first build, simple mental model, limited reproducibility across teams. Excellent for solo developers.
- **Yocto Project / OpenEmbedded**: a layered, recipe-driven build system. Per-layer reuse, multi-machine support, big learning curve, ubiquitous in industry. **This is what Modules 10-12 use.**
- **OpenWrt**: derived from Buildroot, specialised for routers and networking devices. Excellent if your product looks like an AP.
- **Buildroot vs Yocto**: same goal, very different style. Buildroot is the right answer for "one engineer, one board, one product." Yocto is the right answer for "BSP from a vendor, multiple boards, multi-team development."

1.11.3 9.3 Containers / immutable systems

- **balenaOS**, **Rauc-on-Yocto**, **Mender-on-Yocto**: image-based, atomic-update systems built *on top of* Yocto. Worth knowing once you have a real product to update.

1.11.4 Recommendation for this course

Build the same Pi image first with **Buildroot** (one weekend), then again with **Yocto** (one to three weekends). You'll learn Yocto faster after Buildroot has demystified the basics.

1.12 Module 10 -- Yocto concepts

The terminology is the steepest part. Once you have the words, the docs are usable.

1.12.1 10.1 The pieces

- **OpenEmbedded** -- the underlying build framework and recipe collection.
- **Poky** -- the Yocto Project's reference distribution, built from OpenEmbedded.
- **BitBake** -- the build tool. Reads recipes, runs tasks, manages the dependency graph. (Python + custom DSL.)
- **Recipe** (`.bb` files): "how to fetch, configure, compile, and install one piece of software." One recipe per package.
- **Layer**: a directory of recipes (and configuration) packaged together. Layers are *the* unit of reuse. Vendor BSPs are layers (`meta-raspberrypi`, `meta-ti-bsp`, `meta-st-stm32mp`).
- **Machine**: the target hardware. Defined by a `.conf` in a BSP layer (`raspberrypi4-64.conf`).

- **Distro**: the userspace flavour (init system, libc, default package format). `poky` is the default.
- **Image**: the final root filesystem. Defined by an image recipe (`core-image-minimal.bb`).
- **Task**: one step BitBake runs for a recipe (`do_fetch`, `do_compile`, `do_install`, `do_package`).
- `bblayers.conf` and `local.conf`: per-project configuration files in your build directory.

1.12.2 10.2 Mental model of a build

```
You point BitBake at an image recipe (e.g. core-image-minimal).
BitBake walks the dependency tree -> determines every package needed.
For each package's recipe:
   do_fetch -> do_unpack -> do_patch ->
   do_configure -> do_compile -> do_install ->
   do_package -> do_package_write_<format>
Finally an image task assembles everything into a rootfs and bootable
artefact (sdcard.img or wic.img).
```

The first build downloads gigabytes and takes hours. Subsequent builds with **sstate-cache** populated take minutes for small changes.

1.12.3 10.3 The version question

Yocto has **release branches** named after British places (kirkstone, langdale, mickledore, nanbield, scarthgap, ...). **Always pick the current LTS branch** unless you have a specific reason. The current LTS at any given time is documented at https://wiki.yoctoproject.org/wiki/Releases.

1.12.4 10.4 Filesystem hierarchy of a Yocto build

```
your-yocto-project/
|-- poky/                     <- Yocto reference distribution (clone of git)
|-- meta-raspberrypi/         <- BSP layer (clone of git)
|-- meta-openembedded/        <- extra recipes (clone of git)
|-- meta-mycompany/           <- YOUR layer with YOUR recipes
`-- build/
    |-- conf/local.conf       <- machine, distro, package format
    |-- conf/bblayers.conf    <- which layers are active
    |-- downloads/            <- shared download cache (set DL_DIR)
    |-- sstate-cache/         <- shared build cache
    `-- tmp/                  <- per-build outputs (huge)
```

Treat `downloads/` and `sstate-cache/` as shared across all your Yocto projects -- set them outside `build/` so they survive nukes.

1.13 Module 11 -- Your first image on a Raspberry Pi

Concrete steps. Adjust the LTS branch name to what's current when you do this.

1.13.1 11.1 Host requirements

You need a Linux *build host*. Yocto on macOS or Windows directly is not really supported; use a Linux VM or WSL2 if necessary.

- ~100 GiB free disk.

- 16+ GiB RAM strongly recommended.
- Packages: per the Yocto manual's "Required Packages for the Build Host" -- it's a `sudo apt install ...` one-liner you'll copy from the docs.

  Action: read the Yocto Project Quick Build manual end-to-end before typing commands. It's twenty pages. Saves hours of confusion.

1.13.1.1 Realistic first-build costs

"Several hours" undersells it. Set expectations:

- **Downloads (DL_DIR)**: 10-20 GiB on first build for `core-image-minimal` plus a BSP. Goes into `~/yocto/downloads/` if you set it -- otherwise re-downloaded per project.
- **Build outputs (tmp/)**: 50-100 GiB for a small image. This is why the 100 GiB disk requirement matters.
- **sstate-cache**: 10-30 GiB once populated. Worth sharing across projects.
- **Wall-clock time**: on an 8-core laptop with SSD, **4-12 hours** for the first build. Subsequent builds with sstate populated are minutes for small changes.
- **CPU**: pegged near 100% on all cores for stretches; the laptop will be loud.
- **RAM**: BitBake parses thousands of recipes; peak around 4-8 GiB. 16 GiB host RAM is the comfortable floor.

Plan to start a build at the end of one work session and check it the next morning. Don't fight it; this is normal.

1.13.2 11.2 Bring up the tree

```
mkdir ~/yocto && cd ~/yocto

# Pick an LTS branch (replace 'scarthgap' with the current LTS).
git clone -b scarthgap git://git.yoctoproject.org/poky
git clone -b scarthgap git://git.openembedded.org/meta-openembedded
git clone -b scarthgap git://git.yoctoproject.org/meta-raspberrypi

source poky/oe-init-build-env build
```

You're now in `~/yocto/build/`. Edit `conf/local.conf`:

```
MACHINE = "raspberrypi4-64"
DL_DIR ?= "${HOME}/yocto/downloads"
SSTATE_DIR ?= "${HOME}/yocto/sstate-cache"
```

And `conf/bblayers.conf` adds layers:

```
BBLAYERS ?= " \
  ${TOPDIR}/../poky/meta \
  ${TOPDIR}/../poky/meta-poky \
  ${TOPDIR}/../poky/meta-yocto-bsp \
  ${TOPDIR}/../meta-openembedded/meta-oe \
  ${TOPDIR}/../meta-openembedded/meta-python \
  ${TOPDIR}/../meta-openembedded/meta-networking \
  ${TOPDIR}/../meta-raspberrypi \
"
```

1.13.3 11.3 Build

`bitbake core-image-minimal`

Walk away for a few hours on first build.

1.13.4 11.4 Flash and boot

The image is at `tmp/deploy/images/raspberrypi4-64/*.wic.bz2`.

```
bzcat core-image-minimal-raspberrypi4-64.wic.bz2 | sudo dd of=/dev/sdX bs=4M conv=fsync
```

(Replace `/dev/sdX` with your SD card device. **Triple-check** -- this is a destructive command. `lsblk` is your friend.)

Boot the Pi. You should get a login prompt on the HDMI output (or on UART if you've enabled it). Username `root`, no password.

1.13.5 11.5 The QEMU alternative

If you don't have a Pi to hand, set `MACHINE = "qemuarm64"` and use `runqemu` to boot the image in QEMU. Faster iteration loop than re-flashing an SD card. Don't skip Pi hardware indefinitely though -- emulation hides many real-world headaches.

1.14 Module 12 -- Custom layers, recipes, and adding your own software

A real Yocto project is mostly *your own layer* and a few imported ones.

1.14.1 12.1 Create your layer

```
bitbake-layers create-layer ../meta-mkr1000-host
bitbake-layers add-layer ../meta-mkr1000-host
```

You now have:

```
meta-mkr1000-host/
|-- conf/layer.conf
|-- recipes-example/
`-- README
```

1.14.2 12.2 Write a recipe for your daemon

Suppose your Python daemon from Module 8 lives in a git repo `https://example.com/mkr1000-sensord.git`. A minimal recipe:

```
# meta-mkr1000-host/recipes-mkr/mkr1000-sensord/mkr1000-sensord_0.1.bb
SUMMARY = "Bridge between MKR1000 UART and HTTP/metrics"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://LICENSE;md5=<hash here>"

SRC_URI = "git://example.com/mkr1000-sensord.git;protocol=https;branch=main"
SRCREV = "${AUTOREV}"
PV = "0.1+git${SRCPV}"
```

```
S = "${WORKDIR}/git"

inherit setuptools3

RDEPENDS:${PN} += "python3-flask python3-pyserial"
```

This says: fetch the repo, treat it as a `setuptools` Python project, install it, and pull in Flask and pyserial at runtime.

1.14.3 12.3 Add it to your image

Create `recipes-core/images/my-image.bb` in your layer:

```
require recipes-core/images/core-image-minimal.bb

IMAGE_INSTALL:append = " mkr1000-sensord openssh"

IMAGE_FEATURES += "ssh-server-openssh"
```

Build with `bitbake my-image`. The result is the Pi image plus your daemon plus SSH access.

1.14.4 12.4 Devshell: the magic productivity feature

```
bitbake -c devshell mkr1000-sensord
```

Drops you into an interactive shell with the package's source tree and the cross-compilation environment fully set up. Inside, run `./configure`, `make`, or whatever the project needs *by hand*. Excellent for debugging a recipe that's misbehaving.

1.14.5 12.5 The SDK

```
bitbake -c populate_sdk my-image
```

Produces a single shell script (`tmp/deploy/sdk/*.sh`) that, when run on a developer machine, installs a self-contained cross-toolchain + sysroot matching your image. Your daemon's developers now have a deterministic build environment that doesn't require Yocto on their laptops.

> Action: build `my-image`, flash it, SSH in, confirm your daemon starts. Then build the SDK and use it to cross-compile a "hello, world" C program on a different machine and `scp` it over. You're now doing **real** embedded Linux development.

1.14.6 12.6 Common Yocto recipe failure modes

The first dozen recipes you write will fail. Almost always for one of these reasons:

- **`do_fetch` fails: license check.** Symptom: BitBake refuses to fetch with a message about an unspecified license. Fix: in your recipe, set both `LICENSE` (the SPDX name, e.g. `MIT`) and `LIC_FILES_CHKSUM` (one or more `file://...;md5=<hash>` entries). The md5 is a Yocto-specific checksum of a license file *inside the source*; it doesn't change unless the upstream license text changes. Get the right hash by running `bitbake -c fetch <recipe>` once; the error message tells you the expected and actual.
- **`do_unpack` succeeds but `do_compile` cannot find sources.** Cause: `S` (source directory inside `WORKDIR`) is wrong. For git fetches Yocto puts the tree under `${WORKDIR}/git` by default; set `S = "${WORKDIR}/git"` explicitly. For tarballs `S` defaults to `${WORKDIR}/<package>-<version>` which often doesn't match.

- **`do_compile` runs but produces no output.** Cause: the recipe is missing `inherit autotools` (or `cmake`, `meson`, `setuptools3`). Without an `inherit`, BitBake doesn't know how to build the project.
- **`do_install` produces files but `do_package` claims they're unpackaged.** Cause: `FILES:${PN}` doesn't include the paths you installed to. Default `FILES` covers standard FHS paths (`/usr/bin`, `/usr/lib`, etc.) but anything in `/opt`, `/data`, etc. must be added explicitly.
- **`do_package_qa` fails with "installed-vs-shipped" errors.** Cause: files were installed but aren't in any sub-package. Either include them in `FILES:${PN}` or, for files you really mean to drop, list them in `INSANE_SKIP:${PN}`.
- **Recipe builds locally, image build fails with "Nothing RPROVIDES".** Cause: you forgot to `IMAGE_INSTALL:append = " mypkg"` in your image recipe, or your layer is in `bblayers.conf` but the recipe directory's `BBFILES` glob doesn't match the recipe's location.
- **A clean build works, an incremental build is stale.** Cause: sstate cache is reusing an old artefact. Force a rebuild with `bitbake -c cleansstate <recipe>` then rebuild.

When stuck on any of these, the single most powerful command is `bitbake -e <recipe> | less`, which dumps the recipe's complete environment -- every variable, where it was set, every appended value. The answer is almost always somewhere in that output.

1.15 Module 13 -- SoMs and custom carrier boards

When your project outgrows an SBC:

- **System on Module (SoM)**: a small PCB containing SoC + RAM + flash
  - power management, with a board-to-board connector. Examples: Compute Module 4 (Raspberry Pi), Variscite, Toradex, Phytec, Octavo (BeagleBone-on-a-module).
- **Carrier board**: your custom PCB exposing exactly the connectors your product needs, with the SoM plugged in.

Why SoMs win for products:

- The hard parts (DDR layout, BGA fanout, EMI of high-speed nets) are solved by the SoM vendor.
- Your carrier is mostly 2--4 layers and human-soldering-friendly.
- Vendors provide Yocto BSP layers for their SoMs that you `git clone` and base your project on.

Combine with bare-metal Appendix D from the first course (`guide/README.md`): you can absolutely design and order a custom carrier in KiCad. Most of the work is power supplies, connectors, and ESD/EMI -- the SoC itself is hidden inside the SoM.

1.16 Module 14 -- Real-time Linux: PREEMPT_RT, Xenomai, and the MCU boundary

Linux is **not a real-time OS by default**. Worst-case scheduling latency on stock Linux is "milliseconds, usually, but no guarantees." This is fine for HTTP, awful for closed-loop control.

Two ways to add real-time:

1. **PREEMPT_RT** -- a kernel patchset (largely mainlined as of 2024) that converts most kernel locks into preemptible primitives. Pushes worst-case latency from "milliseconds" to "tens of microseconds." Available in Yocto via the linux-yocto `PREFERRED_VERSION` + `LINUX_KERNEL_TYPE = "preempt-rt"`.

2. **Xenomai / RTAI** -- co-kernel approach: a real-time micro-kernel runs underneath Linux, schedules real-time tasks itself, and treats Linux as the idle task. Maximum determinism, niche, complex.

1.16.1 When you actually still want an MCU instead

PREEMPT_RT gets you to roughly tens of microseconds. **If your control loop wants single-digit microseconds or hard deterministic timing**, you keep the MCU. This is why "MCU + Linux paired" is the most common embedded architecture in industry: each side is doing what it's best at.

The boundary is also a software question: how much logic lives in the MCU's firmware versus the Linux side? Push toward the MCU anything timing-critical or that must keep running when the Linux side reboots/crashes. Push toward Linux anything stateful, networked, or user-facing.

1.17 Appendix A -- Glossary

- **AArch64** -- the 64-bit ARM execution state. ARMv8-A and beyond.
- **BSP** -- Board Support Package. The chip- and board-specific code that adapts a generic kernel to a particular hardware.
- **BitBake** -- Yocto's build engine.
- **Buildroot** -- a simpler alternative to Yocto for building embedded Linux distributions.
- **Co-kernel** -- a small real-time kernel running underneath Linux to provide hard real-time guarantees.
- **devicetree** -- a text description of the hardware ("which UART is at which address, which IRQ") consumed by the kernel at boot.
- **devshell** -- a BitBake task that drops you into a shell for a recipe with its build environment.
- **distro** (Yocto) -- the userspace flavour (init, libc, defaults).
- **image** (Yocto) -- a recipe that assembles a complete installable root filesystem.
- **initramfs** -- a small RAM-resident filesystem the kernel mounts before the real rootfs.
- **layer** -- a directory of Yocto recipes and config files; the unit of reuse.
- **machine** (Yocto) -- the target hardware definition.
- **MMU** -- Memory Management Unit; required for mainline Linux.
- **MPU** -- Memory Protection Unit; weaker, found on Cortex-M.
- **OpenEmbedded** -- the underlying framework Yocto builds on.
- **Poky** -- Yocto's reference distribution.
- **PREEMPT_RT** -- the mainlined real-time patchset for the Linux kernel.
- **PRU** -- Programmable Real-time Unit; small co-processors on TI Sitara SoCs (BeagleBone Black has two).
- **rootfs** -- the root filesystem; everything mounted under `/`.
- **recipe** -- a `.bb` file describing how to build one package.
- **SBC** -- Single-Board Computer (e.g. Raspberry Pi).
- **SoC** -- System on Chip; one die containing CPU + peripherals.
- **SoM** -- System on Module; a small PCB with SoC + RAM + flash + power, ready to drop onto a carrier board.
- **sstate-cache** -- BitBake's shared-state cache; reuses compiled outputs across builds.
- **U-Boot** -- "Das U-Boot," the dominant bootloader for embedded ARM Linux.
- **uClinux** -- a fork (now largely re-merged or obsolete) of Linux for MMU-less CPUs.
- **wic** -- Yocto's tool for assembling partitioned disk images; also the file extension (`.wic`, `.wic.bz2`).

1.18 Appendix B -- Yocto / BitBake cheat-sheet

Command	Effect
`source poky/oe-init-build-env build`	Set up the build environment, enter `build/`
`bitbake <recipe>`	Build a recipe and its deps
`bitbake <image>`	Build a full image
`bitbake -c <task> <recipe>`	Run one task: `fetch`, `unpack`, `patch`, `configure`, `compile`, `install`, `package`, `cleansstate`, `devshell`, `populate_sdk`
`bitbake -e <recipe>`	Dump the full environment of a recipe (everything that affected it). Long, indispensable when debugging.
`bitbake-layers show-layers`	List active layers
`bitbake-layers show-recipes`	List all known recipes
`bitbake-layers add-layer <path>`	Add a layer to `bblayers.conf`
`bitbake-layers create-layer <path>`	Scaffold a new layer
`runqemu <machine>`	Boot a qemu image
`oe-pkgdata-util find-path <file>`	Which package owns this file?

Recipe boilerplate cheats:

Variable	Meaning
`SRC_URI`	Where to fetch source (git/http/file)
`SRCREV`	Specific git revision (use `${AUTOREV}` for tip during dev)
`S`	Source directory inside `WORKDIR`
`B`	Build directory (often same as `S`)
`inherit <class>`	Use a `.bbclass`: `autotools`, `cmake`, `meson`, `setuptools3`, `systemd`, ...
`DEPENDS`	Build-time dependencies (other recipes)
`RDEPENDS:${PN}`	Run-time dependencies (other packages)
`IMAGE_INSTALL:append`	Add packages to an image
`FILES:${PN}`	Which files end up in the binary package
`LICENSE` / `LIC_FILES_CHKSUM`	License metadata; checked at parse time

1.19 Appendix C -- Reading list and primary sources

In priority order:

1. **Yocto Project Mega-Manual** -- https://docs.yoctoproject.org/. The single source of truth. Bookmark and search.
2. **"Embedded Linux Systems with the Yocto Project"** by Rudolf J. Streif. Older but still the best long-form tutorial.
3. **Bootlin training materials** -- https://bootlin.com/training/. Free PDFs of full courses. The "Embedded Linux System Development" and "Yocto Project and OpenEmbedded development" decks are excellent.
4. **"Linux Device Drivers, Third Edition"** (Corbet, Rubini, Kroah-Hartman). Free online. Older but the conceptual content is still gold when you have to write a driver.
5. **The Linux kernel's own `Documentation/` tree** -- read what matches the subsystems you touch.

6. **LWN.net** -- weekly kernel news; the best technical writing on Linux kernel topics anywhere. Subscribe.
7. **`devicetree-specification` (devicetree.org)** -- the formal spec; useful when you have to read or write a `.dts`.
8. **Vendor BSP documentation** for whatever SoC you actually use (TI, NXP, ST, Rockchip). Quality varies but always more accurate for the chip than community guesses.

When you're stuck:

- **Yocto IRC** (`#yocto` on Libera.Chat) -- knowledgeable, friendly.
- **Yocto mailing list** -- archived, searchable, the maintainers read it.
- **`bitbake -e <recipe> | less`** -- nine times out of ten the answer is here.